

DOCUMENTACION TECNICA

DOMIDELIS

Sistema de Domicilios Progressive Web App
Arquitectura, Funcionalidades e Instalacion

Versión: 1.0.2

Fecha: Junio 2025

Tecnologias: Node.js, Express, Socket.IO, PWA, Web Push

Autor: Equipo de Desarrollo DOMIDELIS

Tabla de Contenidos

1. Resumen Ejecutivo

2. Arquitectura del Sistema

2.1 Vision General

2.2 Stack Tecnologico

2.3 Diagrama de Arquitectura

3. Estructura del Proyecto

3.1 Arbol de Directorios

3.2 Descripcion de Modulos

4. Backend (API y Servidor)

4.1 Servidor Principal (server.js)

4.2 Autenticacion JWT

4.3 Middleware de Verificacion

4.4 Rutas de Tiendas (/api/tienda)

4.5 Proxy a Google Apps Script

4.6 Socket.IO - Tiempo Real

4.7 Notificaciones Web Push

5. Frontend (PWA Cliente)

5.1 Estructura HTML

5.2 Service Workers

5.3 Manifest JSON

5.4 Modulo de Tiendas (app-tiendas)

6. Funcionalidades por Modulo

6.1 Cliente (index.html)

6.2 Panel de Administracion

6.3 Panel Domiciliario

6.4 Checkout y Confirmacion

6.5 Panel de Tienda (app-tiendas)

6.6 Juego Offline

7. Flujo de Datos

7.1 Flujo de Pedido

7.2 Flujo de Asignacion

8. Seguridad

9. Instalacion y Configuracion

9.1 Requisitos

9.2 Variables de Entorno

9.3 Instalacion Local

10. Despliegue

11. Mantenimiento y Versionado

1. Resumen Ejecutivo

DOMIDELIS es una **Progressive Web App (PWA)** diseñada para la gestion integral de servicios de domicilios en zonas locales. El sistema conecta tres actores principales: **clientes** (quienes realizan pedidos), **domiciliarios** (quienes entregan) y **administradores** (quienes gestionan tiendas, productos y asignaciones), junto con un **portal exclusivo para tiendas** que permite a los establecimientos comerciales administrar sus productos y visualizar sus pedidos.

La aplicacion esta construida sobre una arquitectura **full-stack JavaScript** con un backend en Node.js/Express que actua como proxy hacia Google Apps Script (capa de datos), comunicacion en tiempo real mediante Socket.IO, notificaciones push via Web Push API, y un frontend PWA con capacidad de instalacion en dispositivos moviles, funcionamiento offline parcial y notificaciones nativas.

Características Principales

- Tres roles diferenciados: Cliente, Domiciliario, Admin y Tienda
- Comunicacion en tiempo real via Socket.IO para asignacion de pedidos
- Notificaciones push con VAPID para alertas instantaneas
- PWA instalable en Android e iOS con Service Workers
- Juego offline integrado para momentos sin conexion
- Autenticacion JWT con expiracion de 8 horas
- Panel financiero con graficos y desglose por tienda/metodo de pago
- Sistema de promociones con codigos de descuento

2. Arquitectura del Sistema

2.1 Vision General

El sistema sigue un patron **cliente-servidor** con tres capas diferenciadas:

1. **Capa de Presentacion (Frontend):** HTML5, CSS3 y JavaScript vanilla. Implementa el patron PWA con Service Workers para cacheo de recursos estaticos y experiencia offline.

2. **Capa de Aplicacion (Backend):** Servidor Node.js con Express que expone una API RESTful, gestiona autenticacion JWT, maneja Socket.IO para eventos en tiempo real y coordina notificaciones Web Push.
3. **Capa de Datos:** Google Apps Script (GAS) como backend de datos, accedido via HTTP mediante un Web App desplegado que gestiona lectura/escritura en Google Sheets.

2.2 Stack Tecnológico

Tabla 1. Stack Tecnológico del Sistema

Capa	Tecnologia	Version	Proposito
Runtime	Node.js	20.x	Servidor JavaScript
Framework	Express.js	5.2.1	API REST y servidor HTTP
WebSocket	Socket.IO	4.8.3	Comunicacion en tiempo real
Autenticacion	jsonwebtoken	9.0.3	Tokens JWT para sesiones
Push	web-push	3.6.7	Notificaciones push VAPID
CORS	cors	2.8.6	Politica de origenes cruzados
Configuracion	dotenv	17.4.1	Variables de entorno
HTTP Client	axios	1.15.0	Peticiones a Google Apps Script
Datos	Google Apps Script	-	Backend de datos en Sheets
PWA	Service Workers	-	Cacheo offline
Charts	Chart.js	4.4.0	Graficos en panel admin

2.3 Diagrama de Arquitectura

Figura 1. Diagrama de Arquitectura del Sistema

Componente	Protocolo	Destino	Proposito
Cliente PWA	HTTP + Socket.IO	Node.js Express	Pedidos, catalogo, carrito
Admin Panel	HTTP + Socket.IO	Node.js Express	Gestion, asignaciones, informes
Domiciliario PWA	HTTP + Socket.IO	Node.js Express	Pedidos activos, estados
Tienda PWA	HTTP + JWT	Node.js Express	Productos propios, pedidos
Backend Node.js	Proxy HTTP	Google Apps Script	Persistencia de datos
Backend Node.js	Web Push (VAPID)	Dispositivo Movil	Notificaciones nativas
Google Apps Script	CRUD	Google Sheets	Base de datos

El flujo principal inicia cuando un cliente crea un pedido. El backend recibe la peticion, la reenvia a Google Apps Script para persistencia, emite un evento Socket.IO al panel de administracion y envia una notificacion push al administrador. El admin asigna un domiciliario, lo cual dispara otra notificacion push y un evento Socket.IO al room del domiciliario especifico.

3. Estructura del Proyecto

3.1 Arbol de Directorios

```

C:.
|   .gitignore
|
+---.github/workflows
|       deploy-frontend.yml           # CI/CD para despliegue
|
+---.vscode
|       settings.json                 # Configuración del editor
|
+---app-tiendas                       # Portal independiente para tiendas
|   |   index-tienda.html             # Dashboard de tienda
|   |   login-tienda.html             # Login exclusivo tiendas
|   |   manifest-tienda.json          # Manifiesto PWA de tienda
|   |   sw-tienda.js                  # Service Worker de tienda
|   |
|   +---assets
|   |   +---css/tiendas.css           # Estilos del portal tienda
|   |   +---img/                     # Iconos PWA (72x72 a 512x512)
|   |   +---js/
|   |       config-tienda.js          # Configuración API tienda
|   |       tiendas.js               # Lógica del portal tienda
|   |
+---backend                           # Servidor Node.js
|   |   .env                         # Variables de entorno
|   |   package.json                 # Dependencias del proyecto
|   |   server.js                    # Punto de entrada principal
|   |   railway.json                 # Configuración Railway.app
|   |
|   +---middleware/
|   |       verifyTienda.js          # Middleware JWT para tiendas
|   |
|   +---routes/
|   |       tiendaRoutes.js          # Rutas /api/tienda/*
|   |
|   +---node_modules/                # Dependencias instaladas
|   |
+---docs/                             # Documentación del proyecto
|   |   diagramas/
|   |   docs/
|   |   manuales/
|   |
+---frontend                          # Cliente PWA principal
|   |   admin.html                  # Panel administrativo
|   |   checkout.html              # Formulario de pago
|   |   confirmacion.html          # Página de confirmación
|   |   diagnostico.html           # Página de diagnóstico

```

3.2 Descripción de Módulos

Tabla 2. Descripción de Módulos del Sistema

Modulo	Archivos	Descripción
Cliente	index.html, client.js	Catálogo de tiendas, carrito de compras, selector de zona, popup de promociones
Checkout	checkout.html, checkout.js	Formulario de datos del cliente, selección de método de pago, resumen del pedido
Admin	admin.html, admin.js	Gestión de tiendas, productos, domiciliarios, pedidos activos, historial, informes financieros
Domiciliario	domiciliario.html, domiciliario.js	Pedidos activos, historial de entregas, cambio de estados
Tienda	app-tiendas/*	Portal independiente con login propio para que tiendas vean sus productos y pedidos
Auth	login.html, auth.js, auth-guard.js	Login unificado para admin y domiciliarios con JWT, protección de rutas
Push	push-manager.js, server.js (push)	Suscripción a notificaciones push, envío segmentado por rol
Offline	offline-game.js, offline-game.css	Juego Moto Runner que se activa cuando no hay conexión a internet

4. Backend (API y Servidor)

4.1 Servidor Principal (server.js)

El archivo `server.js` constituye el núcleo del backend. Se encarga de cinco funciones principales: servir el frontend estático (en desarrollo), actuar como proxy hacia Google Apps Script, gestionar conexiones Socket.IO, coordinar notificaciones push y manejar la autenticación JWT.

El servidor se inicializa escuchando en el puerto configurado (por defecto 80) y expone las siguientes capacidades fundamentales:

- **Archivos estaticos:** En modo desarrollo sirve los directorios `frontend/` y `app-tiendas/` via Express static.
- **Proxy API:** Todas las peticiones a `/api` son reenviadas a la URL de Google Apps Script con los parametros y body correspondientes.
- **Socket.IO:** Gestiona rooms por rol (`domiciliario_{id}` , `admin_room`) para entrega selectiva de eventos.
- **Web Push:** Almacena suscripciones en memoria (Map) y distribuye notificaciones segmentadas por rol.
- **Rutas de tienda:** Mount del router `/api/tienda` para el portal de establecimientos.

4.2 Autenticacion JWT

El sistema implementa autenticacion basada en **JSON Web Tokens (JWT)** con las siguientes características:

- **Generacion:** Al iniciar sesion exitosamente, el backend genera un token JWT con payload `{id, nombre, rol}` y expiracion de 8 horas.
- **Roles soportados:** `admin` , `domiciliario` , `tienda`
- **Verificacion:** El middleware `verificarToken` intercepta peticiones con header `Authorization: Bearer <token>`
- **Modo hibrido:** Permite acceso sin token (usuarios publicos) pero bloquea si el token es invalido o expirado (codigo 403).

Proteccion de rutas por accion en el proxy:

Tabla 3. Proteccion de Acciones por Rol

Accion	Rol Requerido	Descripcion
crearTienda, actualizarTienda, eliminarTienda	admin	Gestion completa de tiendas
crearProducto, actualizarProducto, eliminarProducto	admin	Gestion de catalogo
crearDomiciliario, eliminarDomiciliario	admin	Gestion de personal
asignarDomiciliario	admin	Asignacion de pedidos
actualizarEstado	admin o domiciliario	Cambio de estado de pedido
getPedidos, getDomiciliarios	Autenticado (cualquier rol)	Consulta de informacion

4.3 Middleware de Verificacion

El sistema cuenta con dos middlewares especializados:

verificarToken (server.js): Middleware hibrido que permite peticiones publicas pero valida el token cuando se envia. Si el token esta ausente, `req.user` se establece en `null`. Si el token es invalido, retorna HTTP 403.

verifyTienda (middleware/verifyTienda.js): Middleware especifico para el portal de tiendas. Extrae el token del header `Authorization`, verifica la firma con `JWT_SECRET`, comprueba que el rol sea exactamente `tienda` e inyecta los datos del establecimiento en `req.tienda` para uso posterior en las rutas.

4.4 Rutas de Tiendas (/api/tienda)

El archivo `routes/tiendaRoutes.js` define un router Express independiente para el portal de tiendas con tres endpoints principales:

Tabla 4. Endpoints del Portal de Tiendas

Metodo	Ruta	Auth	Descripcion
POST	/api/tienda/login	No	Valida credenciales contra GAS y devuelve JWT con rol tienda
GET	/api/tienda/productos	JWT (tienda)	Obtiene productos filtrados por tiendaId del token
GET	/api/tienda/pedidos	JWT (tienda)	Obtiene pedidos con calculo de subtotal por tienda

Para la ruta de pedidos, el backend realiza un procesamiento adicional: parsea el campo `productosJson`, filtra solo los productos que pertenecen a la tienda autenticada, recalcula el subtotal y devuelve el pedido con el total ajustado al monto correspondiente a esa tienda especifica.

4.5 Proxy a Google Apps Script

El backend actua como **proxy transparente** hacia Google Apps Script. Todas las peticiones autenticadas a `/api` son reenviadas a la URL del script desplegado con los parametros query y body originales. Esto proporciona:

- **Seguridad:** La URL del script y las credenciales de Sheets nunca llegan al cliente.
- **Autenticacion intermedia:** El backend valida JWT antes de permitir operaciones sensibles.
- **Eventos en tiempo real:** Tras reenviar la respuesta, el backend emite eventos Socket.IO y notificaciones push segun la accion realizada.

4.6 Socket.IO - Tiempo Real

Socket.IO gestiona la comunicacion bidireccional entre el servidor y los clientes conectados. El sistema utiliza un modelo de **rooms** para enrutar eventos de forma selectiva:

Tabla 5. Eventos Socket.IO

Evento	Emisor	Receptor	Descripcion
identificar	Cliente	Servidor	Registra al socket en un room segun rol e id
nuevoPedido	Servidor	Admin room	Nuevo pedido creado por un cliente
nuevoPedidoAsignado	Servidor	domiciliario_{id}	Pedido asignado a un domiciliario especifico
estadoActualizado	Servidor	Broadcast	Cambio de estado de un pedido
pedidoAsignado	Servidor	Broadcast	Confirmacion de asignacion con detalle

4.7 Notificaciones Web Push

El sistema utiliza el protocolo **Web Push** con claves VAPID para enviar notificaciones nativas a dispositivos suscritos. Las suscripciones se almacenan en memoria (objeto `Map`) indexadas por endpoint.

Endpoints de Push:

Tabla 6. API de Notificaciones Push

Metodo	Ruta	Descripcion
GET	/api/vapid-public-key	Obtiene la clave publica VAPID para suscripcion
POST	/api/suscripciones	Registra una nueva suscripcion push con rol
POST	/api/suscripciones/eliminar	Elimina una suscripcion por endpoint
POST	/api/enviar-push	Envia notificacion segmentada por roles
GET	/api/suscripciones/estado	Lista suscripciones activas

La funcion interna `enviarPushADomiciliario()` permite notificar directamente a un domiciliario especifico cuando se le asigna un pedido, utilizando el `usuarioId` almacenado en la suscripcion para filtrar.

5. Frontend (PWA Cliente)

5.1 Estructura HTML

El frontend esta compuesto por paginas HTML independientes, cada una representando un modulo funcional completo. Todas las paginas incluyen los metatags PWA necesarios para la instalacion y comportamiento de app nativa:

```
<meta name="theme-color" content="#E63946">
<meta name="mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
<link rel="apple-touch-icon" href="assets/img/icon-152x152.png">
<link rel="manifest" href="manifest.json">
```

El `theme-color` (`#E63946` rojo) se aplica tanto al navegador como a la barra de estado en Android, mientras que `black-translucent` proporciona una experiencia inmersiva en iOS.

5.2 Service Workers

El sistema implementa **dos Service Workers independientes**, uno para el frontend principal y otro para el portal de tiendas, garantizando el aislamiento entre ambos ambitos.

service-worker.js (frontend principal):

- **Estrategia estaticos:** *Cache First* para HTML, CSS, JS e imagenes. Si el recurso esta en cache se sirve inmediatamente; si no, se obtiene de la red y se almacena.
- **Estrategia API:** *Network First* para peticiones a `/api/`. Intenta la red primero y guarda la respuesta en cache. Si falla, sirve desde cache o devuelve un error JSON controlado.
- **Offline fallback:** Si una pagina HTML no esta disponible, redirige al index.

sw-tienda.js (portal tiendas):

- Funcionamiento analogo pero con ambito restringido a `/app-tiendas/`
- Cache independiente con prefijo `tienda-cache-`
- Limpieza automatica de versiones anteriores al activarse

5.3 Manifest JSON

El sistema incluye **dos manifiestos PWA** separados:

manifest.json (aplicacion principal):

- `name:` DOMIDELIS - Domicilios
- `short_name:` DOMIDELIS
- `start_url:` ./index.html
- `display:` standalone
- `theme_color:` #E63946
- `background_color:` #FFFFFF
- Iconos en formatos: 72x72, 96x96, 128x128, 144x144, 152x152, 192x192, 384x384, 512x512

manifest-tienda.json (portal tiendas):

- `name:` DOMIDELIS - Panel de Tienda
- `short_name:` Mi Tienda
- `start_url:` ./index-tienda.html
- Iconos con propiedad `purpose: "any maskable"` para adaptacion a formas del launcher

5.4 Modulo de Tiendas (app-tiendas)

El portal de tiendas es una **aplicacion PWA completamente independiente** con su propio login, service worker y manifiesto. Las tiendas acceden con credenciales especificas (usuario/contrasena validados contra GAS) y reciben un JWT con rol `tienda`.

Funcionalidades disponibles:

- **Tab Productos:** Visualizacion del catalogo propio en formato grid con imagen, nombre, precio y badge. Incluye aviso de que la edicion requiere plan Premium.
- **Tab Pedidos:** Lista de pedidos que contienen productos de la tienda, con filtrado automatico y calculo de subtotal por establecimiento.
- **Instalacion PWA:** Banners de instalacion adaptativos segun el dispositivo (Android/iOS) con instrucciones paso a paso personalizadas.

6. Funcionalidades por Modulo

6.1 Cliente (index.html)

La pagina principal es el punto de entrada para los usuarios finales. Sus componentes principales son:

- **Hero section:** Banner promocional con gradiente, titulo animado, badge de lanzamiento y selector de zona de envio con 16 barrios configurados y precios de envio diferenciados (\$5,000 a \$8,000).
- **Grid de tiendas:** Catalogo de establecimientos con imagen, nombre, descripcion truncada a 2 lineas, rating y badge de categoria.
- **Carrito flotante:** Boton circular con contador que desliza un panel lateral con los items agregados, controles de cantidad y boton de checkout via WhatsApp.
- **Popup promocional:** Modal emergente con sistema de codigos promocionales que se validan y aplican al pedido.
- **Instalacion PWA:** Banners adaptativos para Android (beforeinstallprompt) e iOS (instrucciones manuales), con cierre inteligente que respeta 24h sin mostrar.
- **Animacion de comida:** Efecto visual de emojis (hamburguesas, papas) que explotan al agregar el primer producto al carrito.

6.2 Panel de Administracion

El panel admin es la interfaz de gestion completa del sistema, organizada en secciones navegables mediante tabs horizontales:

Tabla 7. Secciones del Panel Admin

Seccion	Funcionalidades
Tiendas	Crear, editar, eliminar tiendas. Gestion de productos por tienda (CRUD completo). Tabla con imagen, nombre, direccion y acciones.
Pedidos Activos	Tabla en tiempo real de pedidos pendientes. Asignacion de domiciliario con buscador. Cambio de estado. Filtros por tienda y domiciliario.
Domiciliarios	CRUD completo de domiciliarios (nombre, telefono, contrasena, comision). Visualizacion de pedidos asignados por persona.
Historial	Pedidos entregados con seleccion multiple, eliminacion masiva, exportacion a CSV, filtros por tienda/domenar en fecha/metodo de pago.
Informes	Panel financiero con KPIs (ingresos, egresos, balance, pedidos), graficos de barras (Chart.js), diagrama de flujo de dinero, desglose por tienda y metodo de pago.
Pagos Domis	Informe especifico de pagos a domiciliarios con KPIs de comisiones, graficos de desempeno y tabla desglosada por repartidor.

El panel incluye un **banner de activacion de permisos** que solicita acceso a notificaciones y audio, y un **indicador de sistema en vivo** con punto parpadeante verde.

6.3 Panel Domiciliario

Interfaz optimizada para dispositivos moviles que permite a los repartidores:

- **Pedidos Activos:** Lista de pedidos asignados con detalle del cliente, direccion, productos y total. Botones para cambiar estado (pendiente → en camino → entregado).
- **Historial:** Pedidos entregados con filtros por fecha (todos/hoy/ayr/semana), estadisticas de entregas totales y ganancias estimadas.
- **Notificaciones push:** Recepcion instantanea de nuevas asignaciones con sonido de alerta (alerta.mp3) y vibracion.
- **Identificacion Socket.IO:** Al cargar, el domiciliario se identifica con su rol e id para unirse al room correspondiente.

6.4 Checkout y Confirmacion

El flujo de compra consta de dos paginas:

checkout.html: Formulario dividido en dos columnas. La columna izquierda captura datos del cliente (nombre, telefono de 10 digitos, direccion, zona de envio con precio, referencias). La columna derecha muestra el resumen del carrito, selector de metodo de pago (Efectivo, Nequi, Daviplata, Transferencia) y boton de confirmacion via WhatsApp que genera un mensaje preformateado con todo el detalle del pedido.

confirmacion.html: Pagina de agradecimiento con animacion de entrada, instrucciones de pago y enlace directo a WhatsApp para enviar el comprobante.

6.5 Panel de Tienda (app-tiendas)

Portal independiente accesible desde `/app-tiendas/` con autenticacion separada. Las tiendas pueden:

- Iniciar sesion con credenciales propias (validadas contra GAS)
- Visualizar sus productos en grid con imagen, nombre, precio y badge
- Ver pedidos que contienen sus productos con filtrado automatico
- Ver subtotal calculado exclusivamente para su establecimiento

El calculo de subtotales por tienda se realiza en el backend parseando el JSON de productos de cada pedido, filtrando por `tiendaId` y sumando los subtotales individuales de cada item.

6.6 Juego Offline

El sistema incluye un **juego tipo endless runner** (Moto Runner) que se activa automaticamente cuando el dispositivo pierde conexion a internet. Implementado en `offline-game.js` con renderizado en Canvas 2D:

- **Mecanica:** El jugador controla una motocicleta que debe esquivar obstaculos en una carretera con fondo scroll infinito.
- **Controles:** Teclado (flechas/arriba) y tactil (toque en pantalla).
- **Puntuacion:** Sistema de puntos con record almacenado en localStorage.
- **Estados:** Pantalla de inicio, juego activo, game over y boton de reconexion.
- **Activacion:** Se detecta via el evento `offline` del navegador.

7. Flujo de Datos

7.1 Flujo de Pedido

Tabla 9. Flujo de Pedido - Secuencia de 10 Pasos

Paso	Actor	Accion	Destino
1	Cliente	Agrega productos al carrito	localStorage
2	Cliente	Completa checkout	WhatsApp API
3	Admin	Confirma pedido recibido	Panel Admin
4	Admin	Crea pedido en sistema	Backend Proxy
5	Backend	Guarda pedido	Google Sheets
6	GAS	Retorna respuesta con ID	Backend
7	Backend	Emite evento Socket.IO	Panel Admin
8	Backend	Envia push notification	Admin Movil
9	Admin	Asigna domiciliario	Domiciliario
10	Domiciliario	Actualiza estado a entregado	Sistema

7.2 Flujo de Asignacion

Cuando un administrador asigna un domiciliario a un pedido, el backend ejecuta una secuencia de tres pasos asincronos tras responder al cliente:

1. **Socket.IO directo:** Emite `nuevoPedidoAsignado` al room `domiciliario_{id}` con los detalles completos del pedido.
2. **Notificacion push:** Llama a `enviarPushADomiciliario()` que itera suscripciones y envia una notificacion Web Push solo al dispositivo cuyo `usuarioId` coincide.
3. **Broadcast general:** Emite `pedidoAsignado` a todos los sockets para actualizar las vistas de admin en tiempo real.

8. Seguridad

El sistema implementa multiples capas de seguridad:

- **Autenticacion JWT:** Tokens firmados con secreto privado, expiracion de 8 horas, payload con id/nombre/rol. Se transmiten via header Authorization Bearer.
- **Proteccion por rol:** Las acciones criticas (crear, eliminar, asignar) estan protegidas por verificacion de rol en el middleware del proxy.
- **Aislamiento de datos:** Los pedidos de tiendas se filtran en el backend; cada tienda solo ve sus propios productos y subtotales calculados.
- **CORS configurado:** El backend permite origenes cruzados con credenciales y headers Authorization explicitamente permitidos.
- **VAPID:** Las notificaciones push utilizan el protocolo VAPID con par de claves privadas/publicas para autenticacion del servidor.
- **Validacion de entrada:** El checkout limita el telefono a 10 digitos numericos, el nombre a 50 caracteres y la direccion a 60 caracteres.

9. Instalacion y Configuracion

9.1 Requisitos

Tabla 8. Requisitos del Sistema

Componente	Version Minima	Notas
Node.js	18.x	Recomendado 20.x LTS
npm	9.x	Incluido con Node.js
Google Apps Script	-	Web App desplegado con URL publica
Servidor	-	RAM minima 512MB, 1GB recomendado

9.2 Variables de Entorno

El archivo `.env` en la carpeta `backend/` debe contener las siguientes variables:

```
# Servidor
NODE_ENV=production
PORT=80

# JWT
JWT_SECRET=tu_secreto_jwt_minimo_32_caracteres

# VAPID (Web Push)
VAPID_PUBLIC_KEY=clave_publica_vapid
VAPID_PRIVATE_KEY=clave_privada_vapid
VAPID_EMAIL=mailto:tu@email.com

# Google Apps Script (usada en el proxy)
GAS_URL=https://script.google.com/macros/s/TU_SCRIPT_ID/exec
```

Para generar las claves VAPID, ejecutar: `npx web-push generate-vapid-keys`

9.3 Instalacion Local

```
# 1. Clonar o copiar el proyecto
cd backend

# 2. Instalar dependencias
npm install

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Iniciar servidor
npm start

# 5. Servir frontend (en modo desarrollo, el backend sirve frontend/ y app-
tiendas/ automaticamente)
# Abrir http://localhost en el navegador
```

Para desarrollo local con tunnel (acceso externo):

```
# En otra terminal
npx ngrok http 80
```

10. Despliegue

El proyecto incluye configuracion lista para despliegue en **Railway.app** mediante el archivo `railway.json`. El workflow de GitHub Actions en `.github/workflows/deploy-frontend.yml` automatiza el despliegue continuo.

Pasos para despliegue en Railway:

1. Conectar el repositorio GitHub al proyecto Railway.
2. Configurar las variables de entorno en el dashboard de Railway (JWT_SECRET, VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY, VAPID_EMAIL).
3. El archivo `railway.json` especifica el comando de inicio: `node server.js`
4. En produccion (NODE_ENV=production), el backend NO sirve archivos estaticos; el frontend debe desplegarse por separado (GitHub Pages, Netlify, etc.) o configurar un proxy inverso.

Consideraciones para produccion:

- El frontend debe apuntar a la URL de produccion del backend en `config.js`
- Las URLs de Socket.IO deben actualizarse a la URL de produccion
- El Service Worker debe registrarse con la ruta correcta segun el dominio
- Los iconos del PWA deben servirse con HTTPS (requerido para Service Workers y Push)

11. Mantenimiento y Versionado

El versionado del proyecto sigue el formato definido en `package.json` :

```
{
  "name": "backend-domicilios",
  "version": "1.0.2",
  "description": "Backend Domicilios - Proxy + Socket.IO + Push + Frontend estatico"
}
```

Estrategia de cacheo PWA: Los Service Workers utilizan versionado de cache (ej: `tienda-cache-v1`). Al actualizar el frontend, incrementar la version en el SW para forzar la actualizacion en los clientes.

Monitoreo: El endpoint `GET /api/status` retorna el estado del servidor indicando si esta en modo desarrollo o produccion, util para health checks.

Nota sobre Actualizaciones del Service Worker

Los navegadores verifican los Service Workers cada 24 horas. Para forzar una actualizacion inmediata, los usuarios pueden cerrar completamente la pestana y volver a abrirla. El evento `activate` del SW limpia automaticamente las caches de versiones anteriores.